# Using AI to Build as a Junior Developer

## and the many existential questions that come with it

Sneha S

March 7, 2026

# What I'm gonna talk about

- A little about me (2 mins)
- 2 cases: A weekend project & an actual feature (10 mins)
- What's worked for me and what hasn't (4 mins)
- Well okay what now? (2 mins)

# What I'm gonna talk about

- A little about me (2 mins)
- 2 cases: A weekend project & an actual feature (10 mins)
- What's worked for me and what hasn't (4 mins)
- Well okay what now? (2 mins)

This talk, as much as it's about how I'm "using AI", is also about the many feelings and questions that it has brought out in me and my peers. Hopefully we'll get to talk about those things as well.

# What I'm gonna talk about

- A little about me (2 mins)
- 2 cases: A weekend project & an actual feature (10 mins)
- What's worked for me and what hasn't (4 mins)
- Well okay what now? (2 mins)

This talk, as much as it's about how I'm "using AI", is also about the many feelings and questions that it has brought out in me and my peers. Hopefully we'll get to talk about those things as well.

I've done my best to write this presentation by myself. All em-dashes are mine. I took help with the theme though. Available at: `saisneha.com/links/ttt-2026-03`

# About Me

- I'm actually a biologist by training. I graduated from my masters only in 2022.
- I've been pivoting slowly into development over the last 4 years. So this is all actually very new for me.

# The Auth Problem

- Teammates often want to share a KB, dashboard, or toolkit internally/with collaborators, and they need auth. All users don't have Google accounts.

# The Auth Problem

- Teammates often want to share a KB, dashboard, or toolkit internally/with collaborators, and they need auth. All users don't have Google accounts.
- It's a repeated problem; we reimplement hacky solutions again and again, sometimes insecure ones.

# The Auth Problem

- Teammates often want to share a KB, dashboard, or toolkit internally/with collaborators, and they need auth. All users don't have Google accounts.
- It's a repeated problem; we reimplement hacky solutions again and again, sometimes insecure ones.
- I wanted a simple, replicable system for the org and external collaborators

# The Auth Problem

- Teammates often want to share a KB, dashboard, or toolkit internally/with collaborators, and they need auth. All users don't have Google accounts.
- It's a repeated problem; we reimplement hacky solutions again and again, sometimes insecure ones.
- I wanted a simple, replicable system for the org and external collaborators
- Every solution felt too daunting or complex to set up (Keycloak comes to mind; ironically, it might not be as complex to set up now)

# The Initial Prompt

```
Help me build a simple passwordless auth layer for my
documentation site. It should allow users to login
using email OTPs or passkeys.

Use Astro for the frontend. Store all user data in a
SQLite database, but support postgres url as well. Use
FastAPI for the backend. It should support AWS SES and
SMTP. Use an env file to keep all secrets separate,
and put the secrets required in a README.

We'll eventually add Google SSO as well.
```

With this, and some additional prompting, I got something working in under 2 hours. Many major aspects, including passkeys and SMTP support - were one-shotted!.

# And then?

After the MVP worked, I realized I wanted this auth layer **shared** across the org and by external collaborators. I didn't want to have to manage multiple auth layers for each app. (This seems so incredibly basic in retrospect, but I did not think of it at all in the first prompt.)

# And then?

After the MVP worked, I realized I wanted this auth layer **shared** across the org and by external collaborators. I didn't want to have to manage multiple auth layers for each app. (This seems so incredibly basic in retrospect, but I did not think of it at all in the first prompt.)

**So then, I:**

- Had Claude throw together a summary of the repo.

# And then?

After the MVP worked, I realized I wanted this auth layer **shared** across the org and by external collaborators. I didn't want to have to manage multiple auth layers for each app. (This seems so incredibly basic in retrospect, but I did not think of it at all in the first prompt.)

**So then, I:**

- Had Claude throw together a summary of the repo.
- Used the chat interface to discuss the summary and what the point of even building this is, even as a small indie product for mostly myself.

# And then?

After the MVP worked, I realized I wanted this auth layer **shared** across the org and by external collaborators. I didn't want to have to manage multiple auth layers for each app. (This seems so incredibly basic in retrospect, but I did not think of it at all in the first prompt.)

**So then, I:**

- Had Claude throw together a summary of the repo.
- Used the chat interface to discuss the summary and what the point of even building this is, even as a small indie product for mostly myself.
- Made a TODO list of what I wanted to build, and then picked up items one by one when I got free time.

# "Product" Discussion

*Midway, I felt I was getting sucked into a weekend project that would just not be used tomorrow: Should I even continue, or use an existing service, based on my requirements?*

Maybe it was the model's sycophancy but after a bit of back and forth - it said that it does make sense to build this. (I think it only works **because** I wasn't writing it by hand.)
*Total time spent: ~44 hours (5.5 working days)*

GitHub Repo: `saisneha.com/links/gatekeeper`

# PostHog: Background

# PostHog: Background

- Complex monorepo: Nest, Next.js, TypeScript, Flutter

# PostHog: Background

- Complex monorepo: Nest, Next.js, TypeScript, Flutter
- I'd heard of PostHog a lot but never used it

# PostHog: Background

- Complex monorepo: Nest, Next.js, TypeScript, Flutter
- I'd heard of PostHog a lot but never used it
- Needed analytics across multiple stacks

# The Prompt

```
This is a monorepo using nest/next, TS, and flutter.
I want to implement simple product analytics using
posthog, tracking login, each step of the sample
submission flow on mobile, lab flow on web, and more.

First, read through the codebase. Then, ELI5 how
posthog works and how you'd implement posthog in
this mono-repo, and what sdks are available and
how they work, and what env variables you need me
to grab from the dashboard.
```

**Result:** Practically one-shot implementation

# Interesting

**Found a compile issue in PostHog's Flutter SDK**

- Ran into it while testing

# Interesting

**Found a compile issue in PostHog's Flutter SDK**

- Ran into it while testing
- Opened an issue on GitHub, was writing the PR to fix it

# Interesting

**Found a compile issue in PostHog's Flutter SDK**

- Ran into it while testing
- Opened an issue on GitHub, was writing the PR to fix it
- PostHog folks resolved it and closed the issue the same day (that was as good an ad as any for posthog)

# Interesting

**Found a compile issue in PostHog's Flutter SDK**

- Ran into it while testing
- Opened an issue on GitHub, was writing the PR to fix it
- PostHog folks resolved it and closed the issue the same day (that was as good an ad as any for posthog)

*That fact that it was a contribution meant I actually went through lines of code to find the compile issue. I learnt some specific details about Flutter - a language I started coding in just 6 months ago.*

# What It Does Well (For Me)

- **Explains before implementing** - I usually say ELI5 your implementation, and only then have it spit out the plan.

# What It Does Well (For Me)

- **Explains before implementing** - I usually say ELI5 your implementation, and only then have it spit out the plan.
- **One-shots common technical patterns** - WebAuthn/Passkey Support, Google SSO, PostHog Analytics, etc.

# What It Does Well (For Me)

- **Explains before implementing** - I usually say ELI5 your implementation, and only then have it spit out the plan.
- **One-shots common technical patterns** - WebAuthn/Passkey Support, Google SSO, PostHog Analytics, etc.
- I feel a lot less guilt building a project on a new stack because I'm able to (1) check if it's the right project to experiment with the stack much faster, and (2) I can also migrate a lot easier if I need to.

# What else has been great

- I used to be a perfectionist; I'd wait to have a perfect plan before starting to implement. This shift has, in many ways, forced me to just build iteratively.

# What else has been great

- I used to be a perfectionist; I'd wait to have a perfect plan before starting to implement. This shift has, in many ways, forced me to just build iteratively.
- It's also been great for DevOps related debugging. AWS!

# What Hasn't Worked

- **CI/CD:** Wrote a GitHub Action to build docker images on server instead of in CI

# What Hasn't Worked

- **CI/CD:** Wrote a GitHub Action to build docker images on server instead of in CI
- **DRY:** Tends toward repetition, needs nudging to be more DRY, and reviews at the end of each feature

# What Hasn't Worked

- **CI/CD:** Wrote a GitHub Action to build docker images on server instead of in CI
- **DRY:** Tends toward repetition, needs nudging to be more DRY, and reviews at the end of each feature
- **UI consistency:** Needs additional *skills* (and often human eye as well) to be consistent/catch issues

# What's been changing beyond the code for me

- Team members and users - especially those who didn't code - are much faster at feedback and iterations

# What's been changing beyond the code for me

- Team members and users - especially those who didn't code - are much faster at feedback and iterations
- So smaller issues that would have been blocked for days are now getting fixed in hours

# What's been changing beyond the code for me

- Team members and users - especially those who didn't code - are much faster at feedback and iterations
- So smaller issues that would have been blocked for days are now getting fixed in hours
- At the same time, expectations for TaT for fixes is accelerating. That's a tough balance that we're still figuring out.

# What's been changing beyond the code for me

- Team members and users - especially those who didn't code - are much faster at feedback and iterations
- So smaller issues that would have been blocked for days are now getting fixed in hours
- At the same time, expectations for TaT for fixes is accelerating. That's a tough balance that we're still figuring out.
- Older products that we'd built for specific manual analyses are not sufficient - they're not built to sustain heavy querying by an AI agent, for e.g.

# What's been changing beyond the code for me

- Team members and users - especially those who didn't code - are much faster at feedback and iterations
- So smaller issues that would have been blocked for days are now getting fixed in hours
- At the same time, expectations for TaT for fixes is accelerating. That's a tough balance that we're still figuring out.
- Older products that we'd built for specific manual analyses are not sufficient - they're not built to sustain heavy querying by an AI agent, for e.g.
- It's making me think and spend a lot more time on "what is the product" and "what does it mean to learn"

# The Existential Question(s)

What does it even mean to be a "junior" developer anymore?

How does one grow? What will the "market" appreciate 2-3 years from now - with the rapid acceleration in the field?

*Maybe it's judgement and system design/architecture - that's the common refrain we hear. But nothing feels certain anymore, and that is just the reality we're having to reckon with.*

# Thanks for Listening!

Fin.

This presentation is available at: `saisneha.com/links/ttt-2026-03`

`hello@saisneha.com` | `saisneha.com`
`linkedin.com/in/snehasaisneha`
`github.com/snehasaisneha`